13. LES FONCTIONS DE MANIPULATION DE FICHIERS

Notions : FILE, fopen(), fread(), fwrite(), fclose(), open(), read(), write(), close().

Les données stockées en mémoire sont perdues dès la sortie du programme. Les fichiers sur support magnétique (bande, disquette, disque) ou électronique (clé USB) peuvent par contre être conservés, mais au prix d'un temps d'accès aux données très supérieur. On peut distinguer :

- les fichiers séquentiels pour lesquels on accède au contenu dans l'ordre du stockage.
- les fichiers à accès indexé encore appelé accès aléatoire, pour lesquels on peut directement accéder à n'importe quel endroit du fichier.

Le C distingue deux catégories de fichiers :

- Les fichiers « bruts ». Ces fichiers sont des fichiers dits « binaire » dans lesquels
- Les fichiers « bufferisés » utilisant les flux de données standards. Ceux-ci peuvent peuvent être « binaires » ou « texte »,

Les fichiers étant dépendants du matériel, ils ne sont pas prévus dans la syntaxe du C mais par l'intermédiaire de fonctions spécifiques qui diffèrent suivant les systèmes d'exploitation et les compilateurs.

13.1. Fichiers bufferisés

Les opérations d'entrée/sortie sur ces fichiers se font par l'intermédiaire d'un « tampon mémoire » géré automatiquement. Cela signifie qu'une instruction d'écriture n'impliquera pas une écriture physique sur le disque mais dans le tampon, avec une écriture sur le disque uniquement lorsque celui-c sera plein. Les fichiers bufferisés peuvent être utilisés :

- Soit en accès direct, en lecture et écriture, avec tous les éléments de même type et même taille (souvent une structure en format binaire), ceci permettant d'accéder directement à un élément donné (le 10ème, le précédent, l'avant-dernier, etc.).
- Soit en accès séquentiel, avec des éléments de types différents, tous formatés sous forme « texte » (c'est-à-dire qu'un float binaire sera transformé en décimal puis on écrira le caractère correspondant à chaque chiffre), en lecture seule ou écriture seule, ces fichiers seront compréhensibles par n'importe quel autre programme (éditeur de textes, imprimante, etc.). Un tel fichier s'utilise comme l'écran et le clavier, par des fonctions similaires.

Les fichiers sont identifiés par un pointeur sur une structure FILE.

13.1.1. Ouverture et fermeture d'un fichier

Avant de pouvoir lire ou écrire dans un fichier, il faut que celui-ci soit ouvert. C'est ce que réalise la fonction open() qui obéit à la syntaxe suivante :

```
FILE * fopen(char * chemin, char * mode);
```

Cette fonction ouvre le fichier de nom chemin suivant le mode :

- r : lecture seule)
- w : écriture, si le fichier existe il est d'abord vidé
- a : (*append*) écriture à la suite du contenu actuel, création si inexistant
- r+ : lecture et écriture, le fichier doit exister
- w+ : lecture et écriture mais effacement au départ du fichier si existant
- a+ : lecture et écriture, positionnement en fin de fichier si existant, création sinon

On peut rajouter t ou b au mode pour des fichiers texte (gestion des CR/LF, option par défaut) ou binaires, ou encore le définir par défaut en donnant à la variable _fmode la valeur O_TEXT ou O_BINARY.
fopen() retourne un pointeur qui nous servira pour accéder au fichier. En cas d'erreur, le pointeur NULL est retourné, le type d'erreur est donné dans une variable errno, détaillée dans errno.h.
La fonction void perror(char *s mess) permet d'afficher un message correspondant à l'erreur. En général on lui passe le nom du fichier.

On referme un fichier avec la fonction fclose() :

```
int close(FILE * fd);
```

Cette fonction referme le fichier fd, en y recopiant le reste du tampon si nécessaire. Cette fonction est obligatoire pour être sur d'avoir l'intégralité des données effectivement transférées sur le disque. En cas de réussite, elle retourne 0.

13.1.2. Lecture d'un fichier

La lecture dans un fichier se fait par la fonction fread qui obéit à la syntaxe suivante :

```
int fread(void *tampon, int taille, int n, FILE *fd);
```

Cette fonction lit n éléments dont la taille unitaire est taille dans le fichier dont le descripteur est fd et les place dans tampon. En cas de réussite, elle renvoie le nombre d'éléments lus (< n si fin de fichier), sinon elle retourne 0.

13.1.3. Écriture d'un fichier

L'écriture dans un fichier se fait par la fonction fwrite :

```
int fwrite(void *tampon, int taille, int n, File * fd);
```

Cette fonction écrit n élément dont la taille unitaire est taille et contenus dans tampon dans le fichier dont le descripteur est fd. Cette fonction retourne le nombre d'octets écrits ou un nombre différent en cas d'erreur.

13.1.4. Autres fonctions

```
int fflush(FILE * fd)
```

Transfère effectivement le reste du tampon sur disque, sans fermer le fichier (à appeler par exemple avant une instruction qui risque de créer un « plantage »).

```
int fseek(FILE * fd, long combien, int mode)
```

Déplace le pointeur de fichier de combien octets, à partir de : début du fichier (mode=0), position actuelle (mode=1) ou fin du fichier (mode=2). Retourne 0 si tout c'est bien passé. Cette fonction n'est utilisable que si l'on connaît la taille des données dans le fichier (impossible d'aller directement à une ligne donnée d'un texte si on ne connaît pas la longueur de chaque ligne).

```
int feof(FILE * fd)
```

Indique si la fin de fichier est atteinte ou non (0).

- Les fichiers bufferisés permettent aussi des sorties formatées au niveau caractère :

```
char fgetc(FILE * fd)
char fputc(char c, FILE * fd)
```

et même :

```
char ungetc(char c, FILE * fd)
```

qui permet de reculer d'un caractère. Cette fonction correspond donc à {fseek(fd,-1,1);c=fgetc(fd)}.

- Mais aussi au niveau chaîne de caractères :

```
char * fgets(char * s, int max, FILE * fd)
```

Lit une chaîne en s'arrêtant au \n ou à max-1 caractères lus, résultat dans la zone pointée par s, et retour du pointeur s ou NULL en cas d'erreur.

```
char fputs(char * s, FILE * fd)
```

Écrit la chaîne dans le fichier sans ajouter de \n, rend le dernier caractère écrit ou EOF si erreur.

```
int fprintf(FILE * fd, char *format, listearguments)
```

Retourne le nombre d'octets écrits, ou EOF en cas d'erreur. Les \n sont transformés en CR/LF si fichier en mode texte (spécifique PC).

```
int fscanf(FILE * fd, char * format, listeadresses)
```

Retourne le nombre de variables lues et stockées, 0 en cas d'erreur.

Exemple :

```
#include <stdio.h>

int main()
{
  FILE * pfichier;
  char nom_fichier[30], nom_personne[30];
  int  compteur, nb_enregistrements;

  /* Première partie :
     Créer et remplir le fichier */
  printf("Entrez le nom du fichier à créer : ");
  scanf("%s", nom_fichier);
  pfichier = fopen(nom_fichier, "w");
  printf("Nombre d'enregistrements à créer : ");
  scanf("%d", &nb_enregistrements);
  compteur = 0;
  while (compteur < nb_enregistrements)
```

```
  {
    printf("Entrez le nom de la personne : ");
    scanf("%s", nom_personne);
    fprintf(pfichier, "%s\n", nom_personne);
    compteur++;
  }
  fclose(pfichier);

  /* Deuxième partie :
     Lire et afficher le contenu du fichier */
  pfichier = fopen(nom_fichier, "r");
  compteur = 0;
  while (!feof(pfichier))
  {
    fscanf(pfichier, "%s\n", nom_personne);
    printf("NOM : %s\n", nom_personne);
    compteur++;
  }
  fclose(pfichier);
  return 0;
}
```

13.2. Fichiers « bruts »

C'est la méthode la plus efficace et rapide pour stocker et récupérer des données sur fichier, mais aussi la moins pratique. On accède au fichier par lecture ou écriture de blocs d'octets dont la taille est définie par le programmeur. C'est ce dernier qui est chargé de préparer et gérer ses blocs. On choisira en général une taille de bloc constante pour tout le fichier, et correspondant à la taille d'un enregistrement physique (secteur, *cluster*, etc.). On traite les fichiers par l'intermédiaire de fonctions, prototypées dans `stdio.h` (ouverture et fermeture) et dans `unistd.h` (les autres), disponibles sur la plupart des compilateurs.

13.2.1. Ouverture et fermeture d'un fichier

Avant de pouvoir lire ou écrire dans un fichier, il faut que celui-ci soit ouvert. C'est ce que réalise la fonction `open` qui obéit à la syntaxe suivante :

```
int open(const char *chemin, int oflag, mode_t mode);
```

Cette fonction ouvre le fichier de nom `chemin` et retourne un descripteur de fichier (*handle* en anglais) qui permettra de l'identifier dans toutes les autres fonctions (lecture, écriture, déplacement, etc.). En cas d'échec, elle retourne `-1`.
Le paramètre `oflag` précise le mode d'ouverture du fichier :

- `O_RDONLY` : ouverture en lecture seule
- `O_WRONLY` : ouverture en écriture seule
- `O_RDWR` : ouverture en lecture et écriture
- `O_NDELAY` : ouverture non bloquante
- `O_APPEND` : positionnement en fin de fichier avant chaque écriture
- `O_CREAT` : création du fichier si il n'existe pas
- `O_TRUNC` : ouverture avec troncature si le fichier existe
- `O_EXCL` : ouverture exclusive (retourne un code d'erreur si le fichier existe déjà lors d'une création)

Ces constantes peuvent être combinées à l'aide de l'opération OU binaire.
Le paramètre mode définit les droits d'accès au fichier en cas de création (dans les autres cas, il n'est pas nécessaire de positionner ce paramètre). Il peut prendre les valeurs suivantes

- S_ISUID, S_IGID, S_ISVTX : activation respective des bits setuid, setgid et sticky ;
- S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXU : activation respective des droits en lecture, écriture, exécution et lecture/écriture/exécution pour le propriétaire du fichier ;
- S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXG : activation respective des droits en lecture, écriture, exécution et lecture/écriture/exécution pour le groupe du propriétaire du fichier ;
- S_IROTH, S_IWOTH, S_IXOTH, S_IRWXO : activation respective des droits en lecture, écriture, exécution et lecture/écriture/exécution pour les autres.

Là aussi, ces constantes peuvent être combinées à l'aide de l'opération OU binaire.

On referme un fichier avec la fonction close() :

```
int close(int fd);
```

Cette fonction referme le fichier dont le descripteur est fd. En cas de réussite, elle retourne 0, sinon elle retourne -1.

13.2.2. Lecture d'un fichier

La lecture dans un fichier se fait par la fonction read() qui obéit à la syntaxe suivante :

```
ssize_t read(int fd, void *tampon, size_t n);
```

Cette fonction lit n octets dans le fichier dont le descripteur est fd et les place dans un tampon. En cas de réussite, elle renvoie le nombre d'octets transférés, sinon elle retourne -1.
Exemple :

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
   char c;
   int fd;
   fd =  open("/etc/passwd", O_RDONLY);
   if (fd == -1){
     perror("impossible d'ouvrir le fichier\n");
     exit(1);
   }
   while ( read(fd, &c, 1) > 0 )
       putchar(c);
   close(fd);
   return 0;
}
```

13.2.3. Écriture d'un fichier

L'écriture dans un fichier se fait par la fonction write() :

```
ssize_t write(int fd, const void *tampon, size_t n);
```

Cette fonction écrit n octets dans le fichier dont le descripteur est fd à partir d'un tampon. Cette

fonction retourne le nombre d'octets écrits ou `-1` en cas d'erreur.
Exemple :

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
   char tampon[]="Bonjour à tous !";
   int fd;
   fd = open("salut.txt", O_CREAT|O_RDWR, S_IRUSR|S_IWUSR);
   write(fd, tampon, sizeof(tampon));
   close(fd);
   return 0;
}
```

13.2.4. Autres fonctions

```
int eof(int fd)
```

Indique si on se trouve (`1`) ou non (`0`) en fin du fichier.

Le fichier peut aussi être utilisé séquentiellement, c'est-à-dire que le « pointeur de fichier » est toujours placé derrière le bloc que l'on vient de traiter, de manière à pouvoir traiter le suivant. Pour déplacer le pointeur de fichier en n'importe que autre endroit, on appelle la fonction :

```
long lseek(int fd, long combien, int code)
```

Elle déplace le pointeur de fichier de `combien` octets, à partir de : début du fichier si `code`=0, position actuelle si `1`, fin du fichier si `2`. La fonction retourne la position atteinte (en nombre d'octets), `-1L` si erreur.

```
long filelength(int fd)
```

Retourne la taille d'un fichier sans déplacer le pointeur de fichier.

Exemple : **Émulation simplifiée de la commande** `cp`

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#define TAILLE_BLOC 1024

int main(int argc, char * argv[])
{
  int source, destination;
  char buffer[TAILLE_BLOC];
  int nb_lus, nb_ecrits;
  if (argc != 3)
  {
    puts("erreur arguments");
    return 1;
  }
  if ((source=open(argv[1],O_RDONLY))<0)
```

```
  {
    puts("erreur ouverture");
    return 2;
  }
  if ((destination=open(argv[2],O_WRONLY|O_CREAT|
O_TRUNC,S_IREAD|S_IWRITE|S_IEXEC))<0)
  {
    puts("erreur ouverture");
    return 2;
  }
  do
  {
    nb_lus = read(source,( char * )buffer, TAILLE_BLOC);
    if (nb_lus > 0)
      nb_ecrits=write(destination,( char * )buffer, nb_lus);
  }
  while ((nb_lus==TAILLE_BLOC)&&(nb_ecrits>0)) ;
  close(source);
  close(destination);
  return 0;
}
```

13.3. Fonctions propres au VFS (*Virtual File System*) de Linux

Ces fonctions, que l'on appellent « primitives », permettent d'interagir avec le système d'exploitation. Leurs prototypes se trouvent dans les fichiers `<unistd.h>`.

13.3.1. Fonctions de gestion des fichiers

13.3.1.1. Modification du nom du fichier

```
int rename(const char *ancien_nom, const char *nouv_nom)
```

13.3.1.2. Modification des droits d'accès à un fichier

```
int chmod(const char *nom_fichier, mode_t mode)
```

`mode` prend les mêmes valeurs que pour la fonction `open()`.

13.3.1.3. Modification du propriétaire d'un fichier

```
int chown(const char *nom_fichier, uid_t id_utilisateur,
          gid_t id_groupe)
```

13.3.2. Fonctions de gestion des répertoires

13.3.2.1. Création d'un répertoire

Un répertoire `nom_rep` est créé par un appel de la fonction :

```
int mkdir(const char *nom_rep, mode_t mode)
```

mode prend les mêmes valeurs que pour la fonction open().

En cas de succès, la fonction retourne le descripteur associé au répertoire. En cas d'échec, elle retourne la valeur -1 et la variable errno prend l'une des valeurs suivantes :

- EACESS : l'accès n'est pas possible ;
- EEXIST : un fichier de même nom existe déjà ;
- EFAULT : le chemin spécifié n'est pas valide ;
- ENAMETOOLONG : le chemin spécifié comprend un nom trop long ;
- ENOTDIR : l'un des composants du chemin d'accès n'est pas un répertoire ;
- ELOOP : un cycle de liens symboliques est détecté ;
- EMFILE : le nombre maximal de fichiers ouverts par le processus utilisant la fonction est atteint ;
- ENOMEM : la mémoire est insuffisante pour réaliser l'opération ;
- EROFS : le système de gestion de fichiers est en lecture seule ;
- ENOSPC : le système de fichiers est saturé.

13.3.2.2. Destruction d'un répertoire

Un répertoire nom_rep est détruit par un appel de la fonction :

```
int rmdir(const char *nom_rep)
```

En cas d'échec, elle retourne la valeur -1 et la variable errno prend l'une des valeurs suivantes :

- EACESS : l'accès n'est pas possible ;
- EBUSY : le répertoire à détruire est un répertoire courant de processus ou un répertoire racine ;
- EFAULT : le chemin spécifié n'est pas valide ;
- ENAMETOOLONG : le chemin spécifié comprend un nom trop long ;
- ENOENT : le répertoire désigné n'existe pas ;
- ELOOP : un cycle de liens symboliques est détecté ;
- ENOTEMPTY : le répertoire à détruire n'est pas vide ;
- ENOMEM : la mémoire est insuffisante pour réaliser l'opération ;
- EROFS : le système de gestion de fichiers est en lecture seule.

13.3.2.3. Exploration d'un répertoire

Trois fonctions permettent aux processus d'accéder aux entrées d'un répertoire. Ces sont les appels systèmes opendir(), readdir() et closedir(), qui permettent respectivement d'ouvrir un répertoire, de lire une entrée d'un répertoire et de fermer un répertoire. Voici leurs prototypes :

```
DIR *opendir(const char * nom_rep);
```

La fonction ouvre le répertoire nom_rep en lecture et renvoie un descripteur pour le répertoire ouvert. Le type DIR défini dans <dirent.h> représente un descripteur de répertoire ouvert.

```
struct dirent *readdir(DIR * rep);
```

Lit une entrée du répertoire désigné par le descripteur rep et la retourne dans une structure de type struc_dirent. La structure struct_dirent correspond à une entrée de répertoire. Elle comprend les champs suivants :

- long d_ino, le numéro d'inode de l'entrée ;
- unsigned short d_reclen, la taille de la structure retournée ;
- char [] d_name, le nom de fichier de l'entrée considérée.

```
int closedir(DIR * rep);
```

Ferme le répertoire désigné par le descripteur rep.

Exemple :

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>

int main()
{
  char nom[50];
  struct dirent *entree;
  DIR *fd;
  getcwd(nom, 50);
  printf("Mon répertoire courant est %s\n", nom);
  fd = opendir(nom);
  entree = readdir(fd);
  while (entree != NULL)
  {
    printf("Le numéro d'inode de l'entrée est %d et le
           nom de fichier correspondant est %s\n",
           entree->d_ino, entree->d_name);
    entree = readdir(fd);
  }
  closedir(fd);
  return 0;
}
```